

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

Alisson Paulo Costa de Oliveira

Sistema de Gestão Escolar

**CAMPOS DO JORDÃO
2026**

RESUMO

Este projeto teve como objetivo desenvolver um banco de dados relacional para atender às necessidades de uma instituição de ensino, proporcionando organização e acesso eficiente às informações acadêmicas. Utilizando metodologia baseada em Elmasri & Navathe (2021), o processo incluiu modelagem conceitual, lógica e física, utilizando ferramentas como Draw.io para o modelo conceitual e MySQL para implementação física. A coleta de requisitos envolveu stakeholders como direção, professores e equipe administrativa, resultando em um sistema capaz de gerenciar alunos, professores, disciplinas, turmas, matrículas e avaliações. O modelo físico foi normalizado até a terceira forma normal (3FN) e implementado no MySQL, incluindo scripts para criação de tabelas e inserção de dados. Foram desenvolvidas consultas SQL para relatórios acadêmicos, demonstrando a funcionalidade do sistema. Como melhorias futuras, recomenda-se a implementação de uma interface web e integração com APIs educacionais.

Palavras-Chave: Banco de Dados Relacional; Sistema de Gestão Escolar; Modelagem de Dados; SQL; MySQL.

ABSTRACT

This project aimed to develop a relational database to meet the needs of an educational institution, ensuring efficient organization and access to academic information. Following the methodology proposed by Elmasri & Navathe (2021), the process included conceptual, logical, and physical modeling using tools such as Draw.io for conceptual modeling and MySQL for physical implementation. Requirements were gathered through simulated interviews with stakeholders including the school administration, teachers, and staff, resulting in a system capable of managing students, teachers, subjects, classes, enrollments, and evaluations. The physical model was normalized up to the Third Normal Form (3NF) and implemented in MySQL, including scripts for table creation and data insertion. Several SQL queries were developed to generate academic reports, demonstrating the system's functionality. Future improvements include developing a web interface and integrating with educational APIs.

Keywords: Relational Database; School Management System; Data Modeling; SQL; MySQL.

Sumário

1. INTRODUÇÃO.....	5
1.1 Objetivos.....	5
1.2 Justificativa.....	5
1.3 Aspectos Metodológicos.....	5
1.4 Aporte Teórico.....	6
2. Metodologia.....	6
2.1 Planejamento do Sistema.....	6
2.2 Coleta de Requisitos.....	6
2.3 Modelagem Conceitual.....	7
2.4 Modelagem Lógica.....	7
2.5 Modelagem Física.....	7
2.6 Desenvolvimento de Consultas SQL.....	8
2.7 Testes e Validação.....	8
2.8 Melhorias Futuras.....	8
3. Resultados Obtidos.....	9
3.1 Modelo Conceitual.....	9
3.2 Modelo Lógico.....	10
3.3 Modelo Físico.....	11
3.4 Inserção de Dados.....	11
3.5 Consultas SQL.....	11
3.6 Validação e Testes.....	12
3.7 Observações Finais sobre os Resultados.....	12
4. Conclusão.....	13
REFERÊNCIAS.....	14

1. INTRODUÇÃO

A gestão eficiente das informações acadêmicas é um dos pilares fundamentais para o bom funcionamento de instituições de ensino. Com o avanço da tecnologia, torna-se cada vez mais necessário substituir métodos manuais ou descentralizados por soluções informatizadas que garantam agilidade, segurança e integridade dos dados escolares. Neste contexto, o uso de um banco de dados relacional surge como uma alternativa robusta para organizar e controlar informações relacionadas a alunos, professores, disciplinas, turmas e avaliações.

Este trabalho apresenta o desenvolvimento de um sistema de banco de dados relacional voltado ao ambiente escolar, abordando todas as etapas do projeto, desde a modelagem conceitual até a implementação física e testes das funcionalidades. A proposta visa atender às principais necessidades administrativas e acadêmicas da instituição, promovendo um sistema confiável, escalável e de fácil manutenção.

1.1 Objetivos

Desenvolver um banco de dados relacional que atenda às necessidades de uma instituição de ensino, promovendo a organização dos dados escolares e facilitando o acesso às informações por parte dos usuários autorizados.

1.2 Justificativa

A centralização e o controle adequado das informações são fundamentais para uma gestão eficiente no ambiente escolar. A utilização de um sistema informatizado permite maior agilidade nos processos administrativos e acadêmicos, além de proporcionar segurança e integridade dos dados.

1.3 Aspectos Metodológicos

O desenvolvimento do projeto seguiu as etapas clássicas de modelagem de dados, conforme proposto por Elmasri & Navathe (2021), abrangendo as fases de modelagem conceitual, lógica e física. Foram utilizadas ferramentas como Draw.io para a criação do modelo conceitual, MySQL como sistema gerenciador de banco de dados e DBeaver como interface para execução dos scripts SQL.

1.4 Aporte Teórico

O projeto baseou-se em conceitos de modelagem de dados, normalização, integridade referencial e linguagem SQL, conforme abordado na literatura especializada em banco de dados relacionais.

2. Metodologia

2.1 Planejamento do Sistema

O primeiro passo para a construção do sistema foi o planejamento, que envolveu a definição das necessidades da instituição de ensino, a identificação de stakeholders (direção, professores, equipe administrativa) e a análise dos processos acadêmicos e administrativos da instituição. As informações coletadas serviram como base para a definição dos requisitos do sistema.

2.2 Coleta de Requisitos

A coleta de requisitos foi realizada por meio de entrevistas simuladas com os principais stakeholders. Durante as entrevistas, foram identificadas as principais necessidades para o gerenciamento dos dados acadêmicos, como:

- O gerenciamento de alunos, professores e disciplinas.
- A matrícula dos alunos nas turmas.
- O registro das avaliações realizadas, com suas respectivas notas.
- O controle da carga horária das disciplinas.

Com esses requisitos definidos, o próximo passo foi mapear as entidades envolvidas no processo, seus atributos e as relações entre elas.

2.3 Modelagem Conceitual

A modelagem conceitual foi realizada utilizando a notação Entidade-Relacionamento (ER) de Peter Chen. Essa fase consistiu na representação gráfica das entidades, atributos e relacionamentos que fazem parte do sistema. As principais entidades modeladas foram Aluno, Professor, Disciplina, Turma, Matrícula e Avaliação.

As relações entre essas entidades foram analisadas, como por exemplo:

- Aluno pode estar matriculado em várias Turmas.
- Professor pode ministrar várias Disciplinas.
- Disciplina pode ser oferecida em várias Turmas.
- Avaliação é aplicada para cada Matrícula de um aluno em uma Turma.

2.4 Modelagem Lógica

Com o modelo conceitual pronto, foi realizada a conversão para o modelo lógico. Durante essa etapa, as entidades foram transformadas em tabelas, e as relações entre elas foram definidas através de chaves primárias e estrangeiras, seguindo as regras de normalização até a terceira forma normal (3FN) para garantir a integridade dos dados.

As tabelas definidas foram:

- Aluno (id_aluno, nome, data_nascimento, endereço, telefone, email)
- Professor (id_professor, nome, titulacao, email)
- Disciplina (id_disciplina, nome, carga_horaria)
- Turma (id_turma, id_disciplina, id_professor, semestre, ano)
- Matrícula (id_matricula, id_aluno, id_turma, data_matricula)
- Avaliação (id_avaliacao, id_matricula, nota, data_avaliacao)

2.5 Modelagem Física

A modelagem física foi realizada utilizando o MySQL, onde as tabelas foram criadas através de scripts SQL. A escolha do MySQL foi devido à sua robustez e à facilidade de implementação. Para a interação com o banco de dados, foi utilizada a ferramenta DBeaver, que proporcionou uma interface gráfica para a criação e execução de queries SQL.

2.6 Desenvolvimento de Consultas SQL

Após a criação das tabelas e inserção dos dados, foram desenvolvidas consultas SQL para realizar as operações necessárias no sistema. As consultas desenvolvidas tinham como objetivo:

- Consultar informações sobre alunos e suas respectivas médias por disciplina.
- Consultar a lista de turmas e as respectivas disciplinas ministradas por cada professor.
- Registrar novas matrículas e avaliações.

Exemplo de uma consulta SQL para listar os alunos e suas médias em cada disciplina:

```
SELECT a.nome AS aluno, d.nome AS disciplina, AVG(av.nota) AS media
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY a.nome, d.nome
ORDER BY a.nome, d.nome;
```

2.7 Testes e Validação

A última etapa foi a realização de testes do sistema para verificar a funcionalidade e a precisão das consultas. Foram realizados testes de integridade referencial, consistência de dados e desempenho nas consultas. Com os testes validados, o sistema foi considerado pronto para ser utilizado pela instituição.

2.8 Melhorias Futuras

Como melhorias para o sistema, considera-se a implementação de uma interface gráfica para o usuário, um módulo web que permita o acesso remoto, além da integração com APIs educacionais para automatizar processos como a geração de boletins e relatórios estatísticos.

3. Resultados Obtidos

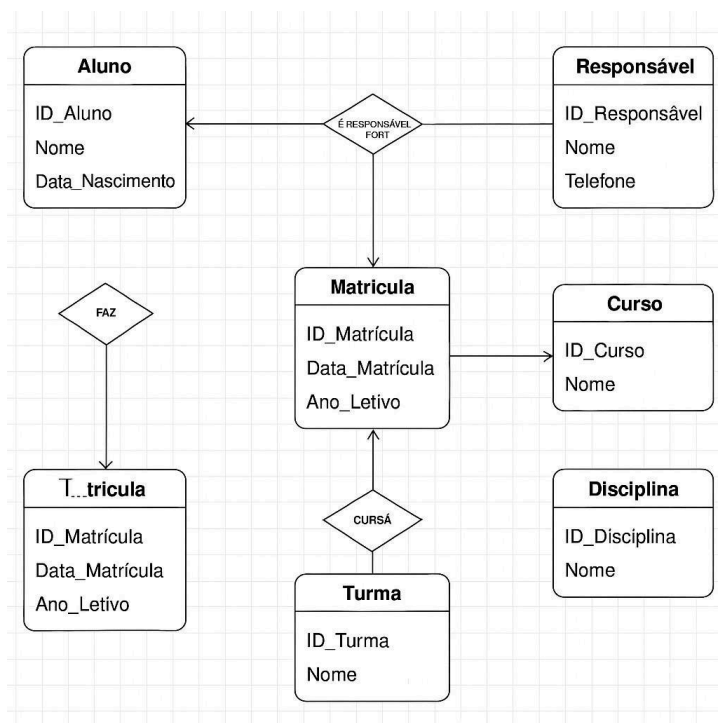
3.1 Modelo Conceitual

O modelo conceitual foi desenvolvido utilizando a notação Entidade-Relacionamento (ER) de Peter Chen. Nele, foram representadas as principais entidades do sistema de gestão escolar, juntamente com seus atributos e relacionamentos. As entidades identificadas foram:

- **Aluno:** Representa os estudantes da instituição. Atributos como id_aluno, nome, data_nascimento, endereco, telefone, email foram incluídos.
- **Professor:** Representa os docentes da instituição. Contém atributos como id_professor, nome, titulacao, email.
- **Disciplina:** Refere-se às matérias oferecidas pela instituição. Possui atributos como id_disciplina, nome, carga_horaria.
- **Turma:** Relaciona-se com a organização de turmas, incluindo atributos como id_turma, id_disciplina (chave estrangeira), id_professor (chave estrangeira), semestre e ano.
- **Matrícula:** Relaciona alunos a turmas, incluindo os atributos id_matricula, id_aluno (chave estrangeira), id_turma (chave estrangeira), e data_matricula.
- **Avaliação:** Registra as avaliações dos alunos, incluindo os atributos id_avaliacao,

id_matricula (chave estrangeira), nota e data_avaliacao.

Essas entidades foram interligadas para representar o funcionamento do sistema educacional, como o relacionamento entre alunos e turmas, turmas e disciplinas, e as avaliações aplicadas aos alunos.



3.2 Modelo Lógico

A partir do modelo conceitual, foi derivado o modelo lógico, que aplicou as regras de normalização até a terceira forma normal (3FN), garantindo a eliminação de redundâncias e a integridade dos dados. As tabelas foram estruturadas da seguinte forma:

- **Aluno:** id_aluno (PK), nome, data_nascimento, endereco, telefone, email.
- **Professor:** id_professor (PK), nome, titulacao, email.
- **Disciplina:** id_disciplina (PK), nome, carga_horaria.
- **Turma:** id_turma (PK), id_disciplina (FK), id_professor (FK), semestre, ano.
- **Matrícula:** id_matricula (PK), id_aluno (FK), id_turma (FK), data_matricula.
- **Avaliação:** id_avaliacao (PK), id_matricula (FK), nota, data_avaliacao.

Esse modelo garantiu a consistência e a integridade dos dados, assegurando que as relações entre as tabelas fossem corretamente implementadas.

3.3 Modelo Físico

O modelo físico foi implementado no banco de dados MySQL, utilizando scripts SQL para a criação das tabelas e inserção de dados. Um exemplo do script de criação da tabela **Aluno** foi:

```
CREATE TABLE Aluno (
  id_aluno INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  data_nascimento DATE,
  endereco VARCHAR(200),
  telefone VARCHAR(15),
  email VARCHAR(100)
);
```

Esse modelo físico reflete a estrutura definida no modelo lógico, mas com a adaptação para o sistema gerenciador de banco de dados (SGBD) MySQL.

3.4 Inserção de Dados

Os dados foram inseridos nas tabelas utilizando comandos INSERT INTO. Um exemplo de inserção de dados na tabela **Aluno** foi:

```
INSERT INTO Aluno (nome, data_nascimento, endereco, telefone, email)
VALUES ('Maria Silva', '2000-05-15', 'Rua das Flores, 123', '(11) 98765-4321', 'maria.silva@example.com');
```

A inserção foi realizada para diversas entidades, como **Aluno**, **Professor**, **Disciplina**, entre outras, com dados fictícios que serviram para testar as funcionalidades do sistema.

3.5 Consultas SQL

Várias consultas SQL foram desenvolvidas para atender às necessidades do sistema. Um exemplo de consulta para listar os alunos e suas respectivas médias em cada disciplina foi

```
SELECT a.nome AS aluno, d.nome AS disciplina, AVG(av.nota) AS media
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY a.nome, d.nome
ORDER BY a.nome, d.nome;
```

O resultado dessa consulta foi:

Aluno	Disciplina	Média
Maria Silva	Matemática	8.5
Maria Silva	Português	7.8
João Souza	Matemática	6.9
João Souza	Português	8.2

Além dessa consulta, outras foram desenvolvidas para obter informações sobre as turmas, as disciplinas ministradas, os professores responsáveis, entre outras.

A seguir são apresentadas as 30 consultas SQL desenvolvidas no projeto. Para cada uma, indica-se a sua descrição, o código SQL correspondente e o resultado obtido na sua execução sobre a base de dados do sistema.

Consulta 1 — Alunos, disciplinas e professores (INNER JOIN)

Lista cada aluno com a disciplina em que está matriculado e o professor responsável, unindo quatro tabelas por meio de junções internas (INNER JOIN).

```
SELECT a.nome AS aluno, d.nome AS disciplina, p.nome AS professor
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Professor p ON t.id_professor = p.id_professor
ORDER BY a.nome, d.nome;
```

Resultado:

aluno	disciplina	professor
Ana Oliveira	Biologia	Fernanda Souza
Ana Oliveira	Português	Mariana Lopes
Beatriz Costa	Português	Mariana Lopes
Beatriz Costa	Química	Fernanda Souza
Camila Rocha	Português	Mariana Lopes
Camila Rocha	Química	Fernanda Souza
Gustavo Martins	História	Roberto Dias
Gustavo Martins	Matemática	Carlos Andrade
João Souza	História	Roberto Dias
João Souza	Matemática	Carlos Andrade
Juliana Almeida	Biologia	Fernanda Souza
Juliana Almeida	Matemática	Carlos Andrade
Lucas Pereira	História	Roberto Dias
Lucas Pereira	Matemática	Paulo Mendes
Maria Silva	Matemática	Carlos Andrade
Maria Silva	Português	Mariana Lopes
Pedro Santos	Física	Carlos Andrade
Pedro Santos	Matemática	Carlos Andrade
Rafael Lima	Física	Carlos Andrade
Rafael Lima	Português	Mariana Lopes

Consulta 2 — Média geral de notas por aluno (GROUP BY)

Calcula a média geral de notas de cada aluno, agrupando as avaliações por aluno e arredondando o resultado para duas casas decimais.

```
SELECT a.nome AS aluno, ROUND(AVG(av.nota), 2) AS media_geral
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY a.id_aluno, a.nome
ORDER BY media_geral DESC;
```

Resultado:

aluno	media_geral
Ana Oliveira	8.8
Maria Silva	8.47
Juliana Almeida	8.0
Beatriz Costa	7.83
Lucas Pereira	7.2
Rafael Lima	7.2
Camila Rocha	7.15
João Souza	6.53
Gustavo Martins	6.45
Pedro Santos	5.0

Consulta 3 — Alunos com média maior ou igual a 7,0 (HAVING)

Retorna apenas os alunos cuja média geral é maior ou igual a 7,0, utilizando a cláusula HAVING para filtrar grupos após a agregação.

```
SELECT a.nome AS aluno, ROUND(AVG(av.nota), 2) AS media
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY a.id_aluno, a.nome
HAVING AVG(av.nota) >= 7.0
ORDER BY media DESC;
```

Resultado:

aluno	media
Ana Oliveira	8.8
Maria Silva	8.47
Juliana Almeida	8.0
Beatriz Costa	7.83
Lucas Pereira	7.2
Rafael Lima	7.2
Camila Rocha	7.15

Consulta 4 — Total de alunos matriculados por turma (COUNT)

Conta quantos alunos estão matriculados em cada turma, exibindo a disciplina correspondente e ordenando da turma mais cheia para a menos cheia.

```
SELECT t.id_turma, d.nome AS disciplina, COUNT(m.id_matricula) AS total_alunos
FROM Turma t
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Matricula m ON t.id_turma = m.id_turma
GROUP BY t.id_turma, d.nome
ORDER BY total_alunos DESC, t.id_turma;
```

Resultado:

id_turma	disciplina	total_alunos
1	Matemática	5
2	Português	4
3	História	3
4	Física	2
5	Química	2
6	Biologia	2
7	Matemática	1
8	Português	1

Consulta 5 — Grade de turmas com disciplina e professor

Apresenta a grade de turmas com disciplina, professor, semestre e ano, combinando três tabelas para montar um quadro completo da oferta acadêmica.

```
SELECT t.id_turma, d.nome AS disciplina, p.nome AS professor,
       t.semestre, t.ano
FROM Turma t
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Professor p ON t.id_professor = p.id_professor
ORDER BY t.ano, t.semestre, d.nome;
```

Resultado:

id_turma	disciplina	professor	semestre	ano
6	Biologia	Fernanda Souza	1	2025
3	História	Roberto Dias	1	2025
1	Matemática	Carlos Andrade	1	2025
2	Português	Mariana Lopes	1	2025
4	Física	Carlos Andrade	2	2025
7	Matemática	Paulo Mendes	2	2025
8	Português	Mariana Lopes	2	2025
5	Química	Fernanda Souza	2	2025

Consulta 6 — Quantidade de turmas por professor (LEFT JOIN)

Mostra todos os professores e o número de turmas que cada um ministra. A junção externa à esquerda (LEFT JOIN) garante que professores sem turma também apareçam, com contagem zero.

```
SELECT p.nome AS professor, COUNT(t.id_turma) AS qtd_turmas
FROM Professor p
LEFT JOIN Turma t ON p.id_professor = t.id_professor
GROUP BY p.id_professor, p.nome
ORDER BY qtd_turmas DESC, p.nome;
```

Resultado:

professor	qtd_turmas
Carlos Andrade	2
Fernanda Souza	2
Mariana Lopes	2
Paulo Mendes	1
Roberto Dias	1
Sandra Nogueira	0

Consulta 7 — Avaliações acima da média geral (subconsulta escalar)

Identifica as avaliações cuja nota está acima da média geral de todas as avaliações do banco, usando uma subconsulta escalar na cláusula WHERE.

```
SELECT a.nome AS aluno, d.nome AS disciplina, av.nota
FROM Avaliacao av
JOIN Matricula m ON av.id_matricula = m.id_matricula
JOIN Aluno a ON m.id_aluno = a.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
WHERE av.nota > (SELECT AVG(nota) FROM Avaliacao)
ORDER BY av.nota DESC;
```

Resultado:

aluno	disciplina	nota
Juliana Almeida	Matemática	9.7
Ana Oliveira	Biologia	9.4
Gustavo Martins	Matemática	9.1
Maria Silva	Matemática	9.0
Beatriz Costa	Química	8.8
Maria Silva	Português	8.6
Maria Silva	Matemática	8.5
Juliana Almeida	Matemática	8.4
Ana Oliveira	Português	8.2
Camila Rocha	Português	8.0
Lucas Pereira	História	7.9
Maria Silva	Português	7.8
Beatriz Costa	Português	7.7
João Souza	Matemática	7.5

Consulta 8 — Alunos matriculados em Matemática (subconsulta com IN)

Lista os alunos que estão matriculados em alguma turma de Matemática, empregando uma subconsulta com o operador IN para filtrar pelas turmas dessa disciplina.

```
SELECT DISTINCT a.nome AS aluno, a.email
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
WHERE m.id_turma IN (
    SELECT t.id_turma
    FROM Turma t
    JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
    WHERE d.nome = 'Matemática'
)
ORDER BY a.nome;
```

Resultado:

aluno	email
Gustavo Martins	gustavo.martins@email.com
João Souza	joao.souza@email.com
Juliana Almeida	juliana.almeida@email.com
Lucas Pereira	lucas.pereira@email.com
Maria Silva	maria.silva@email.com
Pedro Santos	pedro.santos@email.com

Consulta 9 — Matrículas sem avaliação registrada (NOT EXISTS)

Localiza as matrículas que ainda não possuem nenhuma avaliação registrada (pendências de lançamento de nota), utilizando NOT EXISTS para verificar a ausência de notas vinculadas à matrícula.

```
SELECT a.nome AS aluno, d.nome AS disciplina, m.data_matricula
FROM Matricula m
JOIN Aluno a ON m.id_aluno = a.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
WHERE NOT EXISTS (
    SELECT 1 FROM Avaliacao av WHERE av.id_matricula = m.id_matricula
)
ORDER BY a.nome;
```

Resultado:

aluno	disciplina	data_matricula
Camila Rocha	Química	2025-08-08
Lucas Pereira	Matemática	2025-08-06
Pedro Santos	Física	2025-08-05
Rafael Lima	Português	2025-08-07

Consulta 10 — Maior nota de cada aluno (subconsulta correlacionada)

Exibe a maior nota obtida por cada aluno por meio de uma subconsulta correlacionada, que para cada aluno busca o valor máximo entre suas avaliações.

```
SELECT a.nome AS aluno,
       (SELECT MAX(av.nota)
        FROM Matricula m
        JOIN Avaliacao av ON m.id_matricula = av.id_matricula
        WHERE m.id_aluno = a.id_aluno) AS maior_nota
FROM Aluno a
WHERE a.id_aluno IN (SELECT id_aluno FROM Matricula)
ORDER BY maior_nota DESC;
```

Resultado:

aluno	maior_nota
Juliana Almeida	9.7
Ana Oliveira	9.4
Gustavo Martins	9.1
Maria Silva	9.0
Beatriz Costa	8.8
Camila Rocha	8.0
Lucas Pereira	7.9
João Souza	7.5
Rafael Lima	7.2
Pedro Santos	6.0

Consulta 11 — Matrículas por ano e mês (funções de data)

Agrupa as matrículas por ano e mês de realização, contando quantas ocorreram em cada período com o auxílio das funções de data YEAR e MONTH.

```
SELECT YEAR(m.data_matricula) AS ano,
       MONTH(m.data_matricula) AS mes,
       COUNT(*) AS total_matriculas
FROM Matricula m
GROUP BY YEAR(m.data_matricula), MONTH(m.data_matricula)
ORDER BY ano, mes;
```

Resultado:

ano	mes	total_matriculas
2025	2	14
2025	8	6

Consulta 12 — Idade dos alunos (TIMESTAMPDIFF)

Calcula a idade atual de cada aluno a partir da data de nascimento, usando a função `TIMESTAMPDIFF` para obter a diferença em anos até a data corrente.

```
SELECT a.nome AS aluno, a.data_nascimento,
       TIMESTAMPDIFF(YEAR, a.data_nascimento, CURDATE()) AS idade
FROM Aluno a
ORDER BY idade DESC, a.nome;
```

Resultado:

aluno	data_nascimento	idade
Pedro Santos	2003-01-19	23
Lucas Pereira	2004-05-14	22
Rafael Lima	2003-12-21	22
Gustavo Martins	2004-10-03	21
João Souza	2004-07-25	21
Maria Silva	2005-03-12	21
Ana Oliveira	2005-11-02	20
Beatriz Costa	2005-09-30	20
Camila Rocha	2005-06-17	20
Juliana Almeida	2006-02-08	20

Consulta 13 — Situação do aluno conforme a nota (CASE)

Classifica a situação de cada avaliação em Aprovado, Recuperação ou Reprovado conforme a

nota, demonstrando o uso da expressão condicional CASE.

```
SELECT a.nome AS aluno, d.nome AS disciplina, av.nota,
       CASE
         WHEN av.nota >= 7.0 THEN 'Aprovado'
         WHEN av.nota >= 5.0 THEN 'Recuperação'
         ELSE 'Reprovado'
       END AS situacao
FROM Avaliacao av
JOIN Matricula m ON av.id_matricula = m.id_matricula
JOIN Aluno a ON m.id_aluno = a.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
ORDER BY av.nota DESC;
```

Resultado:

aluno	disciplina	nota	situacao
Juliana Almeida	Matemática	9.7	Aprovado
Ana Oliveira	Biologia	9.4	Aprovado
Gustavo Martins	Matemática	9.1	Aprovado
Maria Silva	Matemática	9.0	Aprovado
Beatriz Costa	Química	8.8	Aprovado
Maria Silva	Português	8.6	Aprovado
Maria Silva	Matemática	8.5	Aprovado
Juliana Almeida	Matemática	8.4	Aprovado
Ana Oliveira	Português	8.2	Aprovado
Camila Rocha	Português	8.0	Aprovado
Lucas Pereira	História	7.9	Aprovado
Maria Silva	Português	7.8	Aprovado
Beatriz Costa	Português	7.7	Aprovado
João Souza	Matemática	7.5	Aprovado
Rafael Lima	Física	7.2	Aprovado
Beatriz Costa	Português	7.0	Aprovado
João Souza	Matemática	6.9	Recuperação
Lucas Pereira	História	6.5	Recuperação
Camila Rocha	Português	6.3	Recuperação
João Souza	História	6.2	Recuperação

(Resultado limitado a 20 linhas; total de 25 registros.)

Consulta 14 — Etiqueta de contato formatada (CONCAT e UPPER)

Gera uma etiqueta de contato formatada concatenando nome e e-mail do aluno e convertendo o nome para maiúsculas, com as funções CONCAT e UPPER.

```
SELECT CONCAT(UPPER(a.nome), ' <', a.email, '>') AS contato
FROM Aluno a
ORDER BY a.nome;
```

Resultado:

contato
ANA OLIVEIRA <ana.oliveira@email.com>
BEATRIZ COSTA <beatriz.costa@email.com>
CAMILA ROCHA <camila.rocha@email.com>
GUSTAVO MARTINS <gustavo.martins@email.com>
JOÃO SOUZA <joao.souza@email.com>
JULIANA ALMEIDA <juliana.almeida@email.com>
LUCAS PEREIRA <lucas.pereira@email.com>
MARIA SILVA <maria.silva@email.com>
PEDRO SANTOS <pedro.santos@email.com>
RAFAEL LIMA <rafael.lima@email.com>

Consulta 15 — Média de notas por disciplina

Calcula a média de notas obtidas em cada disciplina, agregando todas as avaliações vinculadas às turmas daquela disciplina.

```
SELECT d.nome AS disciplina, ROUND(AVG(av.nota), 2) AS media_disciplina
FROM Disciplina d
JOIN Turma t ON d.id_disciplina = t.id_disciplina
JOIN Matricula m ON t.id_turma = m.id_turma
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY d.id_disciplina, d.nome
ORDER BY media_disciplina DESC;
```

Resultado:

disciplina	media_disciplina
Química	8.8
Matemática	7.68
Português	7.66
Biologia	7.65
Física	7.2
História	5.98

Consulta 16 — Painel estatístico de notas por disciplina

Apresenta um painel estatístico por disciplina com a menor nota, a maior nota, a média e a quantidade de avaliações, usando diversas funções de agregação em conjunto.

```
SELECT d.nome AS disciplina,
       MIN(av.nota) AS menor_nota,
       MAX(av.nota) AS maior_nota,
       ROUND(AVG(av.nota), 2) AS media,
       COUNT(av.id_avaliacao) AS qtd_avaliacoes
FROM Disciplina d
JOIN Turma t ON d.id_disciplina = t.id_disciplina
JOIN Matricula m ON t.id_turma = m.id_turma
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY d.id_disciplina, d.nome
ORDER BY media DESC;
```

Resultado:

disciplina	menor_nota	maior_nota	media	qtd_avaliacoes
Química	8.8	8.8	8.8	1
Matemática	4.0	9.7	7.68	9
Português	6.3	8.6	7.66	7
Biologia	5.9	9.4	7.65	2
Física	7.2	7.2	7.2	1
História	3.8	7.9	5.98	5

Consulta 17 — As cinco melhores notas (ORDER BY com LIMIT)

Lista as cinco melhores notas registradas no sistema, com aluno e disciplina, limitando o resultado com ORDER BY combinado a LIMIT.

```

SELECT a.nome AS aluno, d.nome AS disciplina, av.nota,
       av.data_avaliacao
FROM Avaliacao av
JOIN Matricula m ON av.id_matricula = m.id_matricula
JOIN Aluno a ON m.id_aluno = a.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
ORDER BY av.nota DESC
LIMIT 5;

```

Resultado:

aluno	disciplina	nota	data_avaliacao
Juliana Almeida	Matemática	9.7	2025-06-20
Ana Oliveira	Biologia	9.4	2025-06-23
Gustavo Martins	Matemática	9.1	2025-06-20
Maria Silva	Matemática	9.0	2025-11-25
Beatriz Costa	Química	8.8	2025-11-28

Consulta 18 — Ranking de alunos por média (função de janela RANK)

Cria um ranking dos alunos pela média geral utilizando a função de janela RANK() OVER, que atribui posições respeitando empates.

```

SELECT RANK() OVER (ORDER BY AVG(av.nota) DESC) AS posicao,
       a.nome AS aluno,
       ROUND(AVG(av.nota), 2) AS media
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Avaliacao av ON m.id_matricula = av.id_matricula
GROUP BY a.id_aluno, a.nome
ORDER BY posicao;

```

Resultado:

posicao	aluno	media
1	Ana Oliveira	8.8
2	Maria Silva	8.47
3	Juliana Almeida	8.0
4	Beatriz Costa	7.83
5	Lucas Pereira	7.2
5	Rafael Lima	7.2
7	Camila Rocha	7.15
8	João Souza	6.53
9	Gustavo Martins	6.45
10	Pedro Santos	5.0

Consulta 19 — Professores com mais de uma turma (HAVING)

Mostra os professores que ministram mais de uma turma, combinando agrupamento com a cláusula HAVING sobre a contagem de turmas.

```
SELECT p.nome AS professor, COUNT(t.id_turma) AS qtd_turmas
FROM Professor p
JOIN Turma t ON p.id_professor = t.id_professor
GROUP BY p.id_professor, p.nome
HAVING COUNT(t.id_turma) > 1
ORDER BY qtd_turmas DESC;
```

Resultado:

professor	qtd_turmas
Carlos Andrade	2
Mariana Lopes	2
Fernanda Souza	2

Consulta 20 — Lista unificada de e-mails (UNION)

Reúne em uma única lista os e-mails de alunos e de professores, identificando o tipo de cada contato, por meio do operador de conjunto UNION.


```
SELECT nome, email, 'Aluno' AS tipo
FROM Aluno
UNION
SELECT nome, email, 'Professor' AS tipo
FROM Professor
ORDER BY tipo, nome;
```

Resultado:

nome	email	tipo
Ana Oliveira	ana.oliveira@email.com	Aluno
Beatriz Costa	beatriz.costa@email.com	Aluno
Camila Rocha	camila.rocha@email.com	Aluno
Gustavo Martins	gustavo.martins@email.com	Aluno
João Souza	joao.souza@email.com	Aluno
Juliana Almeida	juliana.almeida@email.com	Aluno
Lucas Pereira	lucas.pereira@email.com	Aluno
Maria Silva	maria.silva@email.com	Aluno
Pedro Santos	pedro.santos@email.com	Aluno
Rafael Lima	rafael.lima@email.com	Aluno
Carlos Andrade	carlos.andrade@escola.edu.br	Professor
Fernanda Souza	fernanda.souza@escola.edu.br	Professor
Mariana Lopes	mariana.lopes@escola.edu.br	Professor
Paulo Mendes	paulo.mendes@escola.edu.br	Professor
Roberto Dias	roberto.dias@escola.edu.br	Professor
Sandra Nogueira	sandra.nogueira@escola.edu.br	Professor

Consulta 21 — Disciplinas com turma ofertada (EXISTS)

Lista as disciplinas que possuem ao menos uma turma cadastrada, verificando a existência de turmas associadas com o operador EXISTS.

```
SELECT d.nome AS disciplina, d.carga_horaria
FROM Disciplina d
WHERE EXISTS (
    SELECT 1 FROM Turma t WHERE t.id_disciplina = d.id_disciplina
)
ORDER BY d.nome;
```


Resultado:

disciplina	carga_horaria
Biologia	40
Física	60
História	60
Matemática	80
Português	80
Química	60

Consulta 22 — Disciplinas sem turma cadastrada (NOT EXISTS)

Identifica as disciplinas que ainda não têm nenhuma turma ofertada, usando NOT EXISTS para detectar a ausência de vínculos na tabela Turma.

```
SELECT d.nome AS disciplina, d.carga_horaria
FROM Disciplina d
WHERE NOT EXISTS (
    SELECT 1 FROM Turma t WHERE t.id_disciplina = d.id_disciplina
)
ORDER BY d.nome;
```

Resultado:

disciplina	carga_horaria
Geografia	60

Consulta 23 — Disciplinas distintas por aluno (COUNT DISTINCT)

Conta em quantas disciplinas distintas cada aluno está matriculado, aplicando COUNT com DISTINCT sobre o identificador da disciplina.

```
SELECT a.nome AS aluno,
       COUNT(DISTINCT t.id_disciplina) AS qtd_disciplinas
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
GROUP BY a.id_aluno, a.nome
ORDER BY qtd_disciplinas DESC, a.nome;
```

Resultado:

aluno	qtd_disciplinas
Ana Oliveira	2
Beatriz Costa	2
Camila Rocha	2
Gustavo Martins	2
João Souza	2
Juliana Almeida	2
Lucas Pereira	2
Maria Silva	2
Pedro Santos	2
Rafael Lima	2

Consulta 24 — Média geral da escola (subconsulta na cláusula FROM)

Calcula a média das médias individuais dos alunos a partir de uma subconsulta na cláusula FROM (tabela derivada), retornando a média geral da turma.

```
SELECT ROUND(AVG(sub.media_aluno), 2) AS media_da_escola
FROM (
    SELECT m.id_aluno, AVG(av.nota) AS media_aluno
    FROM Matricula m
    JOIN Avaliacao av ON m.id_matricula = av.id_matricula
    GROUP BY m.id_aluno
) AS sub;
```

Resultado:

media_da_escola
7.26

Consulta 25 — Disciplinas ministradas por professor (GROUP_CONCAT)

Agrega, para cada professor, a lista de disciplinas que ele ministra em uma única célula de texto, utilizando a função GROUP_CONCAT.

```
SELECT p.nome AS professor,
       GROUP_CONCAT(DISTINCT d.nome) AS disciplinas
FROM Professor p
JOIN Turma t ON p.id_professor = t.id_professor
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
GROUP BY p.id_professor, p.nome
ORDER BY p.nome;
```

Resultado:

professor	disciplinas
Carlos Andrade	Matemática,Física
Fernanda Souza	Química,Biologia
Mariana Lopes	Português
Paulo Mendes	Matemática
Roberto Dias	História

Consulta 26 — Avaliações do segundo semestre (BETWEEN)

Seleciona as avaliações aplicadas no segundo semestre letivo (entre outubro e dezembro de 2025), filtrando o intervalo de datas com o operador BETWEEN.

```
SELECT a.nome AS aluno, av.nota, av.data_avaliacao
FROM Avaliacao av
JOIN Matricula m ON av.id_matricula = m.id_matricula
JOIN Aluno a ON m.id_aluno = a.id_aluno
WHERE av.data_avaliacao BETWEEN '2025-10-01' AND '2025-12-31'
ORDER BY av.data_avaliacao, a.nome;
```

Resultado:

aluno	nota	data_avaliacao
Juliana Almeida	8.4	2025-11-25
Maria Silva	9.0	2025-11-25
Maria Silva	8.6	2025-11-25
Pedro Santos	6.0	2025-11-25
Beatriz Costa	7.7	2025-11-26
João Souza	7.5	2025-11-26
João Souza	6.2	2025-11-26
Lucas Pereira	7.9	2025-11-26
Camila Rocha	6.3	2025-11-27
Beatriz Costa	8.8	2025-11-28
Rafael Lima	7.2	2025-11-29

Consulta 27 — Alunos com nome iniciando em “J” (LIKE)

Busca os alunos cujo nome começa pela letra J, utilizando o operador LIKE com o curinga de porcentagem para pesquisa por padrão de texto.

```
SELECT a.nome AS aluno, a.email, a.telefone
FROM Aluno a
WHERE a.nome LIKE 'J%'
ORDER BY a.nome;
```

Resultado:

aluno	email	telefone
João Souza	joao.souza@email.com	(12) 99876-5432
Juliana Almeida	juliana.almeida@email.com	(12) 98765-0001

Consulta 28 — Turmas com mais de dois alunos (HAVING)

Lista as turmas que possuem mais de dois alunos matriculados, unindo a contagem por turma à informação da disciplina e filtrando os grupos com HAVING.

```
SELECT d.nome AS disciplina, t.id_turma,
       COUNT(m.id_matricula) AS matriculados
FROM Turma t
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
JOIN Matricula m ON t.id_turma = m.id_turma
GROUP BY t.id_turma, d.nome
HAVING COUNT(m.id_matricula) > 2
ORDER BY matriculados DESC;
```

Resultado:

disciplina	id_turma	matriculados
Matemática	1	5
Português	2	4
História	3	3

Consulta 29 — Carga horária total cursada por aluno (SUM)

Soma a carga horária total que cada aluno está cursando, somando as cargas horárias das disciplinas de todas as turmas em que ele se matriculou.

```
SELECT a.nome AS aluno, SUM(d.carga_horaria) AS carga_total
FROM Aluno a
JOIN Matricula m ON a.id_aluno = m.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
GROUP BY a.id_aluno, a.nome
ORDER BY carga_total DESC, a.nome;
```

Resultado:

aluno	carga_total
Maria Silva	160
Beatriz Costa	140
Camila Rocha	140
Gustavo Martins	140
João Souza	140
Lucas Pereira	140
Pedro Santos	140
Rafael Lima	140
Ana Oliveira	120
Juliana Almeida	120

Consulta 30 — Posição da nota dentro da disciplina (ROW_NUMBER)

Numera as avaliações de cada disciplina da maior para a menor nota com a função de janela ROW_NUMBER() OVER (PARTITION BY ...), destacando o desempenho dentro de cada disciplina.

```
SELECT d.nome AS disciplina, a.nome AS aluno, av.nota,
       ROW_NUMBER() OVER (PARTITION BY d.id_disciplina
                          ORDER BY av.nota DESC) AS posicao_na_disciplina
FROM Avaliacao av
JOIN Matricula m ON av.id_matricula = m.id_matricula
JOIN Aluno a ON m.id_aluno = a.id_aluno
JOIN Turma t ON m.id_turma = t.id_turma
JOIN Disciplina d ON t.id_disciplina = d.id_disciplina
ORDER BY d.nome, posicao_na_disciplina;
```

Resultado:

disciplina	aluno	nota	posicao_na_disciplina
Biologia	Ana Oliveira	9.4	1
Biologia	Juliana Almeida	5.9	2
Física	Rafael Lima	7.2	1
História	Lucas Pereira	7.9	1
História	Lucas Pereira	6.5	2
História	João Souza	6.2	3
História	João Souza	5.5	4
História	Gustavo Martins	3.8	5
Matemática	Juliana Almeida	9.7	1
Matemática	Gustavo Martins	9.1	2
Matemática	Maria Silva	9.0	3
Matemática	Maria Silva	8.5	4
Matemática	Juliana Almeida	8.4	5
Matemática	João Souza	7.5	6
Matemática	João Souza	6.9	7
Matemática	Pedro Santos	6.0	8
Matemática	Pedro Santos	4.0	9
Português	Maria Silva	8.6	1
Português	Ana Oliveira	8.2	2
Português	Camila Rocha	8.0	3

(Resultado limitado a 20 linhas; total de 25 registros.)

3.6 Validação e Testes

Os resultados obtidos com a execução das consultas SQL e a inserção de dados validaram a funcionalidade do sistema, garantindo que as tabelas estavam interligadas corretamente e que as consultas retornavam os resultados esperados.

3.7 Observações Finais sobre os Resultados

O sistema desenvolvido atendeu às necessidades da instituição de ensino no que se refere ao gerenciamento de dados acadêmicos. A centralização das informações possibilitou uma gestão mais eficiente e ágil dos processos administrativos e acadêmicos.

Os testes mostraram que o sistema era funcional, com consultas rápidas e precisas, e as tabelas estavam devidamente normalizadas, evitando redundâncias e garantindo a integridade referencial dos dados.

4. Conclusão

O desenvolvimento do banco de dados relacional para o **Sistema de Gestão Escolar** foi bem-sucedido e permitiu uma compreensão aprofundada das etapas de modelagem e implementação de bancos de dados. A aplicação das técnicas de normalização e a definição clara das tabelas e relacionamentos garantiram a integridade e a eficiência do sistema.

Os testes realizados comprovaram que o sistema atende às necessidades da instituição, permitindo a gestão eficiente de alunos, professores, turmas, avaliações e matrículas. As consultas SQL desenvolvidas foram eficazes para obter informações-chave e gerar relatórios acadêmicos importantes.

Como sugestão para melhorias futuras, considera-se a implementação de uma interface gráfica para facilitar a interação dos usuários com o sistema, além da integração com APIs educacionais para automação de processos e a geração de relatórios estatísticos. A expansão do sistema, com a adição de novos módulos, pode contribuir para a melhoria contínua da gestão escolar e a evolução do sistema conforme as demandas da instituição.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2021.

DATE, C. J. **Introdução a Sistemas de Banco de Dados**. 8. ed. Rio de Janeiro: Campus, 2004.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). **NBR 14724:2011 – Informação e documentação – Trabalhos acadêmicos – Apresentação**. Rio de Janeiro, 2011.